Phase 2, Test Strategy and Plan

Product: NexaPay · **Release:** pre-public-exposure (next quarter) **Author:** Samuel, QA Automation Senior · **Date:** 15 August 2026 · **v1.0**

> This strategy is deliberately narrow. Six risks in the Phase 1 matrix score ≥ 20 and five of those six are security or financial integrity. Effort follows that distribution: the API boundary gets the investment, the UI gets what it uniquely proves, and several conventionally-expected areas are consciously left uncovered with the reasoning stated.

---

1. Quality objectives and targets

Objectives are expressed as things that can be *measured and refuted*. "Improve quality" is not an objective; the rows below are.

#	Objective	Target	How it is measured	Status
O1	No unauthorised party can read or move money	100% of authorisation tests pass	TS-01, TS-01b, TS-02, TS-10b	✗ **failing**
O2	No transfer executes outside documented business rules	100% of P1 boundary tests pass, enforced server-side	TS-04 (10 cases)	○ **unverified**
O3	A retried transfer never debits twice	Replay of an `Idempotency-Key` creates 0 additional records	TS-03	○ **unverified**
O4	What the operator sees equals what the ledger holds	0 cent of divergence, compared in integer cents	TS-08, TS-08b	○ **blocked on Q2**
O5	API responses match a published contract	100% schema conformance	TS-06 (5 cases)	✗ **1 breach**
O6	The critical journey works end-to-end	TS-05 green on every release candidate	TS-05	○ **unverified**
O7	Production incidents are diagnosable	`/health`, `/ready`, `/metrics` up; correlation id echoed	TS-10c, TS-10d	✗ **failing**
O8	The suite is trustworthy	0 flaky tests over 10 consecutive runs; 0 secrets in git	CI history; gitleaks	✓ **met**

O1 and O7 are already failing on verified evidence. O2, O3, O4 and O6 are *unverified* rather than failing, a distinction I hold to throughout, because "we did not test it" and "it is broken" demand different management responses.

ISO/IEC 25010 characteristics targeted

Ranked by what this domain actually needs, not by completeness:

1. **Security**, confidentiality, integrity, non-repudiation, accountability
2. **Functional correctness**, a transfer moves the exact amount, once
3. **Reliability**, maturity, fault tolerance, recoverability (RTO 15 min)
4. **Performance efficiency**, latency and throughput targets in Annexe D
5. **Maintainability**, of the suite itself; a suite nobody can change is dead
6. *Usability / Portability*, acknowledged, not prioritised for this release

---

2. Scope

2.1 In scope, and why

Area	Depth	Justification
Authorisation boundaries (all roles + anonymous)	Deep — API + UI	Highest exposure (R-S1, R-S2, R-S3 = 25/25/20). Cheapest to test, most expensive to miss
Transfer creation rules	Deep — API boundary	The only irreversible action. R03/R01/R02/R04 all re-asserted server-side
Idempotency	Deep — API	Annexe G-B evidences a real double debit
Contract conformance	Medium — 2 documented + 2 undocumented endpoints	Every consumer builds against Annexe E; it already diverges
Ledger↔UI reconciliation	Medium — E2E	Operators authorise J1 based on J3's numbers
Critical journey E2E	Narrow — one path	Proves the chain holds; not a substitute for API coverage
Observability probes	Shallow — existence only	Precondition for meeting any Annexe D target

2.2 Explicitly out of scope, and why

This section matters more than the previous one. Anyone can list what they tested.

Area	Decision	Reasoning
Load & sustained throughput (50 req/s × 10 min)	**Out**	Requires an iso-prod environment; the demo host is shared, unsized and single-instance. Measuring p95 there produces a number that is *worse than no number* because it invites a false conclusion. Needs k6 against a sized environment — scoped, not silently dropped

Area	Decision	Reasoning
Full WCAG 2.1 AA audit	**Out; partially mitigated**	A credible AA audit is days of manual work with assistive tech. Instead, every E2E locator uses `getByRole` + accessible name, so a control that loses its name fails an existing test. That is a tripwire, not an audit, and I say so
Bank settlement path (J5)	**Out — blocked**	No signing secret, no stub, no sandbox (PO question Q5). Carried as an explicit residual risk rather than pretended
Kafka event assertions	**Out — blocked**	No consumer access. Annexe F.5 shows the events exist; I cannot assert on them
File upload (J6)	**Out for this cycle**	Moyenne criticality, no evidence it feeds the ledger. Exploratory session only. Revisit if it becomes an ingestion path
EN/FR i18n	**Out — exploratory only**	Faible criticality. Automating string coverage is high-maintenance, low-yield
Unit / component tests	**Out of QA's hands**	Owned by the dev team. I define the exit criterion (§4), not the tests
DR / RTO / RPO drills	**Out**	Requires infrastructure control I do not have. Flagged for the platform team
Multi-currency	**Out — undefined**	The domain statement requires it; the product exposes only a `USD` default. Nothing to test until specified

---

3. Risk-based approach

Coverage is allocated from the Phase 1 matrix, not evenly.

Exposure band	Risks	Coverage rule	Effort
≥ 20 (blocker)	R-S1, R-S2, R-S3, R-B1, R-B2	Automated, P1, in the blocking CI gate. Negative + boundary mandatory	**~55%**
15–19 (mitigate)	R-B3, R-C1, R-C2, R-T1, R-O1, R-S4	Automated where testable; escalated where it needs a decision	~25%
9–14 (monitor)	R-T2, R-T3, R-T4, R-O2, R-C3	Guard-rail test or documented residual risk	~15%

Exposure band	Risks	Coverage rule	Effort
≤ 8 (accept)	R-B4, R-O3	Noted, not automated	~5%

Concretely: 55% of the effort sits on 5 risks, and zero effort goes to the spending-category chart.

---

4. Test levels and responsibilities

Level	Owner	What it must prove	Exit criterion	Status
Unit	Dev	Money arithmetic, rounding, PIN hashing, date/timezone conversion	≥ 80% line coverage on `transfer-svc`; **100% on monetary arithmetic**	▯ no visibility
Component	Dev	`transfer-svc` validation in isolation, deps stubbed	Every Annexe C rule has a rejecting test	▯ no visibility
Integration	Dev + QA	transfer-svc ↔ bank-adapter, ↔ Kafka, webhook verification	Contract tests both directions	⚠ blocked (no access)
System / API	**QA — this suite**	Rules enforced at the HTTP boundary; authorisation; contract	All P1 API tests green	▯ 9 failing, 14 unverified
E2E	**QA — this suite**	The critical journey holds across the whole chain	TS-05 + TS-08 green	∘ unverified
Exploratory	QA	The unknown unknowns	2 × 90-min charters per release	✔ 1 session done

The zero-visibility rows are a finding. I cannot see whether unit tests exist. If they do not, this suite is carrying load it should not: a rounding bug belongs in a 2 ms unit test, not a 300 ms HTTP round-trip. That is in the two-week plan (§13).

Deliberate anti-pattern avoided: the temptation with a rich E2E tool is to push everything to E2E. I have one E2E journey plus four narrow UI checks. Every business rule is asserted at the API level, where the failure message is precise and the runtime is milliseconds.

---

5. Test types and estimated effort

Effort is for **one engineer, one cycle** (≈ the 6 h budget plus what I would ask for next).

Type	In this cycle	Effort	Next cycle	Rationale
Functional — API	24 automated tests	**2.0 h**	+8	Highest yield per hour on this product
Security — authz	6 tests (anon, cross-role, direct-URL)	**1.0 h**	+pen-test	Where the top risks are
Contract / schema	5 tests, Ajv, contract vs observed	**0.7 h**	+consumer-driven	Divergence already found
Data consistency	6 tests (referential, partition, reconciliation)	**0.8 h**	+ledger recon job	Ledger-specific, high value, cheap
E2E functional	1 journey + 4 checks	**0.8 h**	+2 journeys	Kept small on purpose
Exploratory	1 × 90 min charter	**0.5 h**	2/release	Found DEF-001 — no automation would have
Performance	Latency guard-rail only	**0.2 h**	k6 suite, 2 d	Needs iso-prod
Accessibility	Role-based locators as tripwire	*(absorbed)*	axe-core + manual, 3 d	Honest partial
Resilience / chaos	None	0	Timeout/retry, 2 d	Needs fault injection
Total	**32 tests**	**≈ 6 h**		

Worth noting on the exploratory line: DEF-001, unauthenticated access to the user directory, was found in the first fifteen minutes by pointing a client at an endpoint without credentials. No amount of scripted coverage against an *authenticated* client would have surfaced it, because every scripted test would have dutifully logged in first.

---

6. Test environments

6.1 What exists vs what is needed

Characteristic	Demo (today)	Required for a launch decision
Isolation	□ shared, mutable	Per-run tenant or reset hook
Data reset	□ none	Seed + teardown endpoint, or nightly restore
Parity with prod	□ unknown sizing	Iso-prod for any perf claim
External deps	□ real/absent	Bank sandbox or service virtualisation
Observability	□ no probes	`/health`, `/ready`, `/metrics`
Data provenance	□ diverges from docs	Documented, versioned seed

6.2 Consequences I have already absorbed

Because the environment is shared and unresettable:

- Write-path tests run **serialised** (`workers: 1`). Slower: but a parallel double-debit test would corrupt its own premise.
- Every mutation uses a **unique `Idempotency-Key`** so runs do not collide.

- Assertions are **invariant-based**: never pinned to a specific record.
- **No test deletes data**: despite R09 describing deletion, irreversible on a shared environment, and it contradicts Annexe D's immutable-retention rule anyway (C-01 in the contradiction list).

---

7. Test data strategy

Concern	Approach
Read fixtures	The 15 seeded transactions, 3 users, 3 contacts. Captured verbatim in `src/fixtures/recorded/` with a provenance MANIFEST
Write data	Generated per test: UUID idempotency keys, unique notes. Nothing shared between tests
Boundary data	Derived from Annexe C rules, not from the seed — `999999` / `1000000` cents for R03, 60/61-char notes for R04, 5/6-digit PINs for R02
Anonymisation	Not required — the seed is synthetic. **But:** the real risk is the reverse. If this suite is ever pointed at a production-like dataset, `redact.ts` already strips `pin`, `password`, `token` and `Authorization` from every report attachment. Built in now, before it is needed
Secrets	Environment variables only. `.env` git-ignored, `.env.example` carries empty values, CI reads from the encrypted store, gitleaks runs on every push
Determinism	Fixed boundary values; no `Math.random()` in assertions; timestamps computed relative to run time, never hard-coded

Test-data gap found: the Manager account (u2) owns **zero contacts**, so the Manager's transfer journey cannot be completed with the seeded data. Reported as R-B4, a defect in the fixtures, not in the product, and exactly the kind of thing that silently reduces coverage.

---

8. Dependencies, mocks and service virtualisation

My policy, in priority order:

1. **Test against the real thing wherever reachable.** The `live` mode is the default and is what any meaningful run uses.
2. **Never mock the system under test.** Mocking NexaPay's own API to test NexaPay is circular. The replay server exists for *availability*, never to make a test pass.
3. **Virtualise only across a trust boundary you do not own**, the bank adapter, once a contract exists.
4. **A fallback must never manufacture a verdict.** `tools/mock-server.mjs` replays only what was observed; everything else returns `501 not_recorded` and the dependent tests **skip**. This is why the recorded run reports 14 skips instead of 14 invented passes.

Dependency	Policy now	Target
NexaPay API	Live	Live
External bank	Untestable	WireMock stub from a contract
Kafka	Untestable	Embedded broker in integration tests
Webhook sender	Untestable	Signed-event generator (needs Q5)

---

9. Entry and exit criteria

Entry, API / System level

- Environment reachable: `/health` returning 200 *(currently)*
- Seed data loaded and documented
- Credentials for all three roles provisioned
- Build tagged and traceable to a commit

Exit, per level

Level	Exit criteria
Unit / Component	≥ 80% coverage on `transfer-svc`, **100% on monetary arithmetic**; every Annexe C rule has a rejecting test
API / System	**100% of P1 tests pass — no exceptions, no waivers.** ≥ 90% of P2. Zero open Blocker or Critical. Every skip has a documented reason
E2E	Critical journey green on 3 consecutive runs (flakiness gate). Reconciliation shows 0 cent divergence
Release	All of the above, plus: residual risks formally accepted **by name** by the PO, and a rollback tested

The P1 gate has no waiver clause. On a payments product, “we shipped with a known authorisation bypass because the date slipped” is not a trade-off anyone should be allowed to make informally.

---

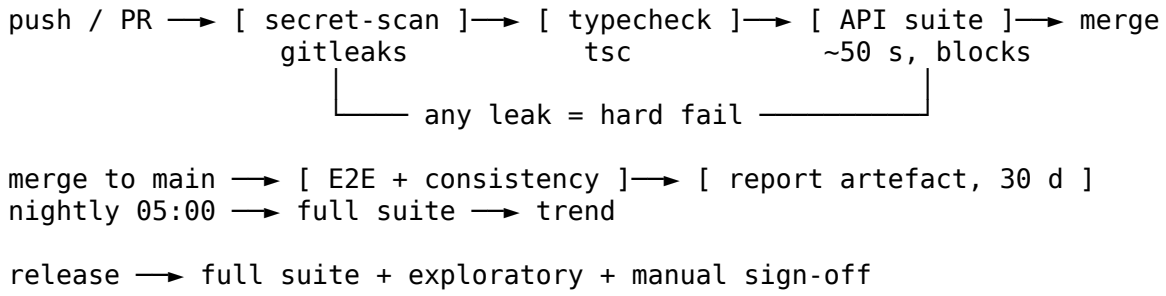
10. Regression strategy

Layer	Trigger	Runtime	Blocking
Smoke (`@smoke`)	Every deploy	< 30 s	
API suite	Every push / PR	~50 s	
E2E + consistency	Merge to main, nightly	~5 min	for release
Full + exploratory	Release candidate	~1 day	

Regression selection: every defect found gets a test *before* the fix, and that test joins the P1 suite permanently. The suite grows by evidence, not by intuition.

Flakiness policy: a test that fails intermittently is quarantined within 24 h and either fixed or deleted within a week. A tolerated flaky test trains the team to ignore red, which costs more than the coverage it provides. Target: **zero** quarantined tests at release.

11. Automation strategy, CI and quality gates



The gates, and what each is for

Gate	Rule	Why
G0 — secrets	Any credential in git = hard fail	Cheapest, most damaging class
G1 — types	`tsc --noEmit` clean	Catches contract drift before a single request
G2 — P1 API	100% pass	The blocker risks live here
G3 — contract	Schema conformance	Protects every consumer
G4 — E2E	Critical journey green ×3	Release-blocking, not PR-blocking

E2E deliberately does **not** block a PR. A browser test that fails for environmental reasons and blocks ten developers costs more than it saves; it blocks the release instead, which is where the decision actually matters.

---

12. Reporting and metrics

Per run: pass/fail/skip by priority, defects by severity, P1 coverage of risks ≥ 20, and, separately, the *skip count with reasons*.

Trend: escaped-defect rate, mean time to detect, flakiness rate, P1 gate pass rate.

Two anti-metrics I will not report, because both reward the wrong behaviour:

- **Raw test count.** The brief says it: 10 well-designed tests beat 40 flat ones. Counting tests incentivises padding.
- **Line coverage as a quality claim.** 100% coverage of code that never validates an amount tells you nothing. I report *risk coverage*, which of the ≥ 20 risks has a passing test, because that is the question a release manager is actually asking.

---

13. Residual risks and mitigation

Risks I am **not** closing this cycle, stated so they can be accepted knowingly rather than discovered later.

Risk	Residual exposure	Proposed mitigation	Owner
Bank settlement path untested (J5)	**High**	Signing secret + stub → automate forged/replayed/out-of-order events	Platform + QA
No load validation	**High**	k6 against iso-prod before public exposure	Perf + QA
`POST /transfers` unverified from my runner	**High**	Run the 14 written tests live — one command	QA
Unit/component coverage unknown	Medium	Publish coverage to the same dashboard	Dev
No DR / RTO drill	Medium	Game-day before launch	Platform
Accessibility partial	Medium	axe-core + assistive-tech pass	QA + Design
Timezone semantics undefined	Medium	Specify, then boundary-test around midnight Paris	PO + Dev
No test-data isolation	Medium	Per-run tenant; unlocks parallelism	Platform
Multi-currency undefined	Low *(rising)*	Specify before the first non-USD corridor	PO

With two more weeks

1. **Days 1–2**, run the 14 transfer tests live; close the verification gap.
2. **Days 3–5**, a resettable test tenant. Unblocks parallelism and removes three medium risks at once. Highest leverage item on the list.
3. **Days 6–8**, webhook coverage, once Q5 is answered.
4. **Days 9–10**, k6 load suite against iso-prod.
5. **Days 11–12**, consumer-driven contract tests (Pact) so the Annexe E divergence cannot recur silently.
6. **Days 13–14**, accessibility pass and a mutation-testing spike on `transfer-svc` to find out whether the unit tests actually assert anything.

Continues in `docs/03-test-design.md`.